

Cryptanalyse

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction à la cryptanalyse : histoire et classification des attaques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

La cryptanalyse est l'étude des méthodes permettant de retrouver, en tout ou partie, l'information protégée par un système cryptographique (message clair, clé) sans disposer légitimement de la clé de déchiffrement. Elle est le pendant offensif de la cryptographie, et les deux disciplines progressent ensemble : un algorithme n'est considéré fiable qu'après avoir résisté longtemps à un examen cryptanalytique public et intensif (principe de Kerckhoffs, vu au module *Cryptographie*).

2. Repères historiques

- **Antiquité à Renaissance** : chiffrement de César, chiffres de substitution ; cassés par analyse fréquentielle dès le IXe siècle par le savant arabe Al-Kindi.
- **XVIe siècle** : chiffre de Vigenère, longtemps considéré incassable (« le chiffre indéchiffrable »), cassé au XIXe siècle par Kasiski et Babbage.
- **Seconde Guerre mondiale** : cryptanalyse de la machine Enigma par les équipes polonaises puis britanniques (Bletchley Park, Alan Turing) — un tournant pour la cryptanalyse moderne et la naissance de l'informatique.
- **Ère moderne** : cryptanalyse différentielle et linéaire contre les chiffrements par blocs (DES), attaques mathématiques contre RSA (factorisation), attaques par canaux auxiliaires.

3. Classification des attaques selon les informations disponibles

Type d'attaque	Information disponible à l'attaquant
Ciphertext-only (texte chiffré seul)	uniquement des messages chiffrés
Known-plaintext (texte clair connu)	un ou plusieurs couples (clair, chiffré)
Chosen-plaintext (texte clair choisi)	l'attaquant peut faire chiffrer des textes de son choix
Chosen-ciphertext (texte chiffré choisi)	l'attaquant peut faire déchiffrer des textes chiffrés de son choix (hors le message cible)

Plus l'attaquant dispose de capacités (de ciphertext-only à chosen-ciphertext), plus le modèle est puissant, et plus un algorithme résistant à un modèle fort est considéré robuste.

4. Classification des attaques selon la méthode

- **Attaques statistiques** : exploitation des propriétés statistiques du texte clair (fréquence des lettres, des mots) ou du chiffrement.
- **Attaques par force brute (exhaustive search)** : test systématique de toutes les clés possibles.
- **Attaques structurelles/mathématiques** : exploitation d'une faiblesse mathématique de l'algorithme (factorisation, logarithme discret, biais différentiels ou linéaires).
- **Attaques par canaux auxiliaires (side-channel)** : exploitation d'informations physiques fuitées par l'implémentation (temps d'exécution, consommation électrique) plutôt que de l'algorithme lui-même.
- **Attaques par ingénierie sociale ou implémentation** : exploitation d'erreurs d'implémentation (générateur aléatoire faible, réutilisation de clé, mode opératoire mal choisi) plutôt que de l'algorithme mathématique — en pratique, la source la plus fréquente de compromission de systèmes cryptographiques réels.

5. Notion de sécurité et de coût d'une attaque

Un système est dit sûr non pas parce qu'il est mathématiquement incassable dans l'absolu, mais parce que le coût de la meilleure attaque connue (temps de calcul, mémoire, nombre de messages requis) dépasse très largement ce qu'un attaquant réaliste peut mobiliser. La sécurité est donc **relative et évolutive** : un algorithme sûr aujourd'hui peut devenir vulnérable avec de nouvelles techniques d'attaque ou une puissance de calcul accrue (ex. menace future de l'informatique quantique sur RSA, abordée au chapitre 5).

6. Méthodologie générale d'une cryptanalyse

1. Comprendre précisément l'algorithme et son mode d'utilisation (mode opératoire, gestion des clés).
2. Identifier le modèle d'attaque réaliste (quelles informations l'attaquant peut-il obtenir ?).
3. Rechercher des biais statistiques, structurels ou d'implémentation exploitables.
4. Évaluer le coût de l'attaque (complexité temporelle, données nécessaires) et le comparer à un budget attaquant réaliste.
5. Le cas échéant, implémenter une preuve de concept sur un exemple contrôlé.

| À retenir

- La cryptanalyse et la cryptographie évoluent en miroir : un algorithme n'est validé que par une résistance prouvée à la cryptanalyse publique.
- Le modèle d'attaque (ciphertext-only à chosen-ciphertext) détermine la puissance réaliste de l'attaquant.
- En pratique, la majorité des compromissions viennent d'erreurs d'implémentation, pas de failles mathématiques dans l'algorithme lui-même.

Chapitre 2 — Cryptanalyse des chiffrements classiques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Rappel : chiffrement de César et substitution mono-alphabétique

Le chiffre de César décale chaque lettre d'un nombre fixe de positions (clé de 0 à 25). Une substitution mono-alphabétique généralise le principe en remplaçant chaque lettre par une autre selon une permutation quelconque de l'alphabet.

2. Analyse fréquentielle

Chaque langue possède une distribution caractéristique de fréquence des lettres (en français : $E \approx 14,7\%$, $A \approx 7,6\%$, $S \approx 7,9\%$, etc.). Une substitution mono-alphabétique conserve cette distribution en la « déplaçant » : la lettre la plus fréquente du texte chiffré correspond très probablement à la lettre la plus fréquente de la langue.

Méthode

1. Compter la fréquence de chaque lettre dans le texte chiffré.
2. Comparer au profil de fréquence connu de la langue.
3. Émettre des hypothèses de correspondance pour les lettres les plus fréquentes.
4. Affiner par l'analyse des digrammes/trigrammes fréquents (« ES », « LE », « ENT » en français) et par le sens du texte partiellement déchiffré.
5. Itérer jusqu'à obtenir un texte cohérent.

Pour le chiffre de César spécifiquement, il suffit de tester les 25 décalages possibles (attaque exhaustive triviale, espace de clé minuscule).

3. Le chiffre de Vigenère et son cassage

Le chiffre de Vigenère applique un décalage de César différent à chaque position, répété selon une clé de longueur L . Il résiste à l'analyse fréquentielle directe (la distribution des lettres chiffrées est « aplatie »), mais reste cassable :

Étape 1 — Trouver la longueur de la clé

- **Méthode de Kasiski** : repérer les répétitions de séquences de lettres identiques dans le texte chiffré ; la distance entre deux occurrences est probablement un multiple de la longueur de clé. Le PGCD des distances observées donne une estimation de L .
- **Indice de coïncidence** : mesure statistique de la probabilité que deux lettres tirées au hasard dans un texte soient identiques. Un texte en langue naturelle a un indice de coïncidence élevé ($\approx 0,0778$ en français) ; un texte purement aléatoire a un indice bas ($\approx 1/26 \approx 0,0385$). En découpant le texte chiffré en L sous-suites (une par position de clé) et en testant plusieurs valeurs de L , on retient la valeur pour laquelle l'indice de coïncidence moyen des sous-suites se rapproche de celui de la langue.

Étape 2 — Casser chaque sous-alphabet

Une fois L connue, chaque sous-suite est un simple chiffre de César indépendant, cassable par analyse fréquentielle classique (chapitre précédent).

4. Autres chiffrements classiques

- **Chiffre de Playfair** (digrammes) : résiste mieux à l'analyse fréquentielle simple mais reste vulnérable à l'analyse des digrammes fréquents.
- **Chiffre de transposition** (permutation des lettres sans substitution) : la distribution de fréquence des lettres est conservée à l'identique (contrairement à la substitution), ce qui trahit immédiatement ce type de chiffrement et oriente vers une cryptanalyse par recherche d'anagrammes/permutations.

5. Ce que cette cryptanalyse historique enseigne

- Toute redondance statistique du texte clair qui « survit » au chiffrement est exploitable — principe qui reste valable pour évaluer des schémas modernes mal conçus (ex. mode ECB, chapitre TP3).
- Un espace de clé trop petit (chiffre de César : 25 clés) rend l'attaque exhaustive triviale, quelle que soit la qualité apparente de l'algorithme.
- La confusion (rendre la relation clé/texte chiffré complexe) et la diffusion (répartir l'influence de chaque bit du clair sur tout le chiffré), concepts formalisés plus tard par Claude Shannon, sont directement motivées par la nécessité de résister à l'analyse fréquentielle.

| À retenir

- L'analyse fréquentielle casse toute substitution mono-alphabétique.
- Le chiffre de Vigenère se casse en deux temps : détermination de la longueur de clé (Kasiski, indice de coïncidence), puis cryptanalyse fréquentielle de chaque sous-alphabet.
- Ces méthodes historiques posent les bases conceptuelles (confusion, diffusion, exploitation de redondance) de la cryptanalyse moderne.

Chapitre 3 — Cryptanalyse des chiffrements symétriques modernes

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Attaque par force brute (exhaustive key search)

Pour un chiffrement par bloc à clé de n bits, l'attaque par force brute nécessite en moyenne $2^{(n-1)}$ essais. C'est pourquoi la taille de clé est le premier paramètre de sécurité : DES (56 bits) est aujourd'hui cassable par force brute en quelques heures avec du matériel spécialisé, ce qui l'a rendu obsolète au profit d'AES (128, 192 ou 256 bits), hors de portée de la force brute avec la technologie actuelle et prévisible.

2. Cryptanalyse différentielle

Introduite publiquement par Biham et Shamir (fin des années 1980) contre DES. Principe :

1. Choisir une paire de textes clairs présentant une différence fixe (souvent un XOR précis).
2. Chiffrer les deux textes et observer la différence entre les chiffrés correspondants.
3. Certaines différences de sortie apparaissent avec une probabilité plus élevée que si le chiffrement était parfaitement aléatoire : ce biais statistique permet de déduire de l'information sur des bits de la clé (ou de sous-clés de tour).
4. Répéter sur de nombreuses paires pour affiner la déduction.

DES s'est révélé partiellement vulnérable à cette technique mais sa conception (notamment le choix des boîtes-S) avait déjà anticipé et limité cette attaque — la NSA connaissait apparemment la cryptanalyse différentielle avant sa publication académique.

3. Cryptanalyse linéaire

Introduite par Matsui (1993). Principe : rechercher des approximations linéaires (combinaisons XOR de bits d'entrée, de sortie et de clé) vraies avec une probabilité significativement différente de $1/2$. Une approximation suffisamment biaisée, combinée à un grand nombre de couples clair/chiffré connus, permet de retrouver des bits de clé par vote statistique (lemme du pilage/*piling-up lemma*).

4. Pourquoi AES résiste-t-il mieux ?

AES a été conçu (concours public NIST, choix de Rijndael en 2001) en intégrant explicitement des critères de résistance à la cryptanalyse différentielle et linéaire dès sa conception (structure des boîtes-S optimisée, nombre de tours dimensionné avec marge de sécurité). Aucune attaque académique connue à ce jour ne casse AES pleine puissance plus rapidement qu'une recherche exhaustive proche de l'optimum théorique.

5. Attaques structurelles sur les modes opératoires

Un algorithme de bloc robuste (AES) peut néanmoins être compromis par un **mode opératoire** mal choisi ou mal utilisé :

Mode	Faiblesse exploitable
ECB (Electronic Codebook)	des blocs de clair identiques produisent des blocs chiffrés identiques → motifs visibles (voir TP3)
CBC sans authentification	vulnérable aux attaques par <i>padding oracle</i> si un serveur révèle si le padding est valide (voir TP3)
Réutilisation de nonce/IV (CTR, GCM)	annule la sécurité du chiffrement en flux résultant, permet de retrouver le XOR de deux messages clairs

Ces attaques ne cassent pas l'algorithme de bloc lui-même : elles exploitent un mauvais usage, ce qui souligne l'importance, en cryptographie appliquée, de suivre des schémas éprouvés (AES-GCM, ChaCha20-Poly1305) plutôt que de composer soi-même bloc + mode.

6. Attaques par compromis temps-mémoire

Les **tables arc-en-ciel (rainbow tables)** illustrent un compromis entre temps de calcul et espace de stockage pour inverser une fonction à sens unique (utilisées historiquement contre des hachages de mots de passe non salés — approfondi au chapitre 4).

À retenir

- La taille de clé fixe la borne théorique de résistance à la force brute ; DES est aujourd'hui cassable, AES ne l'est pas avec la technologie actuelle.
- Cryptanalyse différentielle et linéaire exploitent des biais statistiques internes à la structure de l'algorithme et ont directement influencé la conception d'AES.
- La grande majorité des compromissions pratiques de chiffrements symétriques modernes viennent d'un mauvais mode opératoire ou d'une mauvaise gestion des nonces/IV, pas d'une faille dans l'algorithme de bloc lui-même.

Chapitre 4 — Attaques sur les fonctions de hachage

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Rappel des propriétés attendues

Une fonction de hachage cryptographique doit garantir : la **résistance à la préimage** (impossible de retrouver un message à partir de son empreinte), la **résistance à la seconde préimage** (impossible de trouver un autre message ayant la même empreinte qu'un message donné), et la **résistance aux collisions** (impossible de trouver deux messages distincts ayant la même empreinte).

2. Le paradoxe des anniversaires et l'attaque par collision

Pour une empreinte de n bits, trouver une collision par force brute « naïve » coûterait en théorie 2^n essais. Le **paradoxe des anniversaires** montre qu'il suffit en réalité d'environ $2^{(n/2)}$ essais pour trouver une collision avec une probabilité significative (par analogie : dans un groupe de 23 personnes, la probabilité que deux partagent un anniversaire dépasse 50 %, bien moins que les 365 attendus intuitivement).

Conséquence directe : une fonction de hachage de n bits n'offre qu'une sécurité de $n/2$ bits contre les collisions, ce qui explique pourquoi MD5 (128 bits, soit 64 bits de sécurité en collision) est aujourd'hui totalement cassé en pratique, et pourquoi SHA-1 (160 bits) est déconseillé depuis la démonstration d'une collision pratique en 2017 (attaque « SHAttered »).

3. Attaques historiques marquantes

- **MD5** : collisions pratiques démontrées dès 2004-2005, exploitées ensuite pour forger de faux certificats numériques (affaire du malware Flame, 2012).
- **SHA-1** : collision pratique démontrée par Google et le CWI en 2017, accélérant la migration généralisée vers SHA-2/SHA-3.
- **SHA-2 (SHA-256, SHA-512) et SHA-3** : à ce jour, aucune attaque pratique ne remet en cause leur sécurité ; ce sont les standards recommandés.

4. Attaque par extension de longueur (length extension attack)

Les fonctions basées sur la construction de Merkle-Damgård (MD5, SHA-1, SHA-2) sont vulnérables à l'attaque par extension de longueur : connaissant $H(\text{message})$ et la longueur de `message` (mais pas `message` lui-même), un attaquant peut calculer $H(\text{message} || \text{padding} || \text{extension})$ pour une extension de son choix, **sans connaître** `message`.

Conséquence pratique : construire une authentification de message par simple concaténation $H(\text{clé} || \text{message})$ est dangereux — un attaquant peut forger un MAC valide pour un message étendu sans connaître la clé. La construction **HMAC** (chapitre Cryptographie) a été spécifiquement conçue pour neutraliser cette attaque.

5. Hachage de mots de passe : attaques et contre-mesures

Attaque	Principe	Contre-mesure
Force brute / dictionnaire	tester des mots de passe candidats et comparer les empreintes	politique de mots de passe robustes, limitation du nombre d'essais
Table arc-en-ciel	tables précalculées d'empreintes pour inverser rapidement un hachage non salé	salage (valeur aléatoire unique par mot de passe, ajoutée avant hachage)
Calcul massivement parallèle (GPU/ASIC)	les fonctions de hachage génériques (SHA-256) sont très rapides à calculer, donc favorables à l'attaquant	fonctions spécialement conçues pour être lentes : bcrypt, scrypt, Argon2 (facteur de coût réglable)

6. Méthodologie d'évaluation d'une fonction de hachage

1. Vérifier la taille de sortie (borne théorique de sécurité en collision = $\text{taille}/2$).
2. Vérifier l'absence d'attaques publiées de préimage/collision pratiques.
3. Vérifier le contexte d'usage : une fonction sûre pour l'intégrité de fichiers (SHA-256) ne l'est pas nécessairement pour le hachage de mots de passe sans adaptation (facteur de travail, salage).

À retenir

- Le paradoxe des anniversaires divise par deux, en bits, la sécurité effective d'une fonction de hachage contre les collisions.
- MD5 et SHA-1 sont cassés en pratique pour les collisions et ne doivent plus être utilisés pour un usage sécuritaire.
- Le hachage de mots de passe exige des fonctions dédiées (bcrypt, scrypt, Argon2), volontairement lentes et salées — jamais une fonction générique seule (SHA-256 brut).

Chapitre 5 — Cryptanalyse des systèmes asymétriques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

| 1. RSA : rappel du principe et hypothèse de sécurité

La sécurité de RSA repose sur la difficulté supposée de **factoriser un grand nombre entier** produit de deux grands nombres premiers ($n = p \times q$). Aucun algorithme classique connu ne factorise efficacement (en temps polynomial) un nombre suffisamment grand — c'est une hypothèse calculatoire, pas une preuve mathématique absolue.

2. Attaques par factorisation

- **Force brute / essai de division** : totalement impraticable pour des clés de taille réaliste (2048 bits ou plus).
- **Algorithmes sous-exponentiels** : le *General Number Field Sieve* (GNFS) est l'algorithme classique le plus efficace connu pour factoriser de grands nombres ; sa complexité, bien que sous-exponentielle, reste hors de portée pour des clés RSA de 2048 bits avec la technologie actuelle. C'est pourquoi la taille de clé recommandée a augmenté avec le temps (512 bits cassé dès 1999, 768 bits cassé en 2009, 1024 bits considéré fragilisé, 2048 bits recommandé aujourd'hui).
- **Menace quantique** : l'algorithme de Shor, exécuté sur un ordinateur quantique suffisamment puissant, factoriserait un grand nombre en temps polynomial — casserait RSA (et le logarithme discret, donc Diffie-Hellman/ECC classiques). Cette menace motive la standardisation en cours d'algorithmes post-quantiques (ex. NIST PQC : ML-KEM/Kyber, ML-DSA/Dilithium).

3. Attaques exploitant une mauvaise implémentation de RSA

Ces attaques ne cassent pas le problème mathématique sous-jacent mais exploitent des erreurs pratiques, bien plus fréquentes en réalité :

Attaque	Condition d'application
Exposant public e trop petit sans padding ($e = 3$)	message court, chiffré sans padding correct (OAEP), permet une racine cubique directe
Modules partagés / clés faibles	deux clés RSA différentes partagent accidentellement un facteur premier → PGCD des deux modules révèle instantanément la factorisation
Attaque de Wiener	exposant privé d choisi trop petit par rapport au module → récupération de d par développement en fractions continues
Attaque de Coppersmith	conditions particulières sur des bits connus de p , q ou du message
Réutilisation de nonce / mauvaise génération aléatoire	générateur pseudo-aléatoire faible lors de la génération des nombres premiers → cas réel documenté sur des équipements embarqués bon marché

4. Diffie-Hellman et logarithme discret

La sécurité de Diffie-Hellman (et d'ElGamal) repose sur la difficulté du **problème du logarithme discret** : étant donné g , p et $g^x \bmod p$, retrouver x est supposé difficile. Les meilleures attaques classiques connues (*index calculus*) ont une complexité comparable à la factorisation RSA pour une taille de groupe équivalente, d'où des recommandations de taille de clé similaires.

Attaque pratique notable : l'attaque *Logjam* (2015) a montré que de nombreux serveurs réutilisaient les mêmes petits groupes premiers standardisés pour Diffie-Hellman, permettant un précalcul mutualisé rendant l'attaque de logarithme discret praticable contre des groupes de 512 bits — illustration qu'un paramètre partagé et sous-dimensionné affaiblit tous les systèmes qui le réutilisent.

5. Courbes elliptiques (ECC)

Le problème du logarithme discret sur courbe elliptique (ECDLP) est considéré plus difficile, à taille de clé égale, que son équivalent sur les entiers : une clé ECC de 256 bits offre un niveau de sécurité comparable à une clé RSA de 3072 bits, pour un coût de calcul et une taille de clé bien moindres — ce qui explique l'adoption croissante d'ECC (ECDSA, X25519) dans les protocoles modernes (TLS 1.3, SSH).

6. Démarche générale de cryptanalyse d'un système asymétrique

1. Vérifier les paramètres utilisés (taille de clé, courbe/groupe standardisé et non « maison »).
2. Rechercher des erreurs de génération de clé (facteurs partagés, aléa faible).
3. Vérifier l'usage correct du padding (OAEP pour le chiffrement RSA, PSS pour la signature).
4. Ne considérer la factorisation/logarithme discret « brut » que comme dernier recours théorique : en pratique, l'immense majorité des compromissions viennent des points 1 à 3.

À retenir

- La sécurité de RSA et Diffie-Hellman repose sur des hypothèses calculatoires (factorisation, logarithme discret), pas sur une preuve d'impossibilité absolue.
- Les attaques réellement exploitées en pratique ciblent presque toujours une mauvaise implémentation (aléa faible, paramètres partagés, padding absent) plutôt que l'algorithme mathématique lui-même.
- La menace de l'informatique quantique (algorithme de Shor) motive la transition en cours vers la cryptographie post-quantique.

Chapitre 6 — Attaques par canaux auxiliaires et attaques pratiques

Cryptanalyse

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Principe des attaques par canaux auxiliaires (side-channel)

Une attaque par canal auxiliaire n'exploite pas de faiblesse mathématique de l'algorithme, mais une **information physique fuitée par son implémentation** : temps d'exécution, consommation électrique, rayonnement électromagnétique, comportement du cache processeur, messages d'erreur. Ces attaques rappellent que la sécurité d'un système cryptographique dépend autant de son implémentation que de sa conception mathématique.

2. Attaques temporelles (timing attacks)

Si le temps d'exécution d'une opération cryptographique dépend des bits secrets manipulés (ex. une comparaison de mot de passe qui s'arrête au premier octet différent), un attaquant mesurant précisément ce temps peut déduire de l'information sur le secret, octet par octet.

Contre-mesure : comparaisons en temps constant (*constant-time comparison*), qui parcourent systématiquement toute la donnée indépendamment du résultat intermédiaire.

3. Attaques par oracle de padding (padding oracle)

Décrites en détail en TP3 : si un système révèle (même indirectement, par un code d'erreur différent ou un temps de réponse différent) si un texte chiffré déchiffré présente un padding valide selon un schéma comme PKCS#7 en mode CBC, un attaquant peut déchiffrer progressivement l'intégralité du message sans connaître la clé, en interrogeant le système bloc par bloc, octet par octet. Cette classe d'attaque (Vaudenay, 2002) a compromis en pratique de nombreuses implémentations TLS/SSL (ex. attaque *POODLE*, 2014).

4. Attaques par analyse de consommation électrique et électromagnétique

- **SPA (Simple Power Analysis)** : observation directe de la trace de consommation, révélant parfois la séquence d'opérations effectuées (ex. distinguer un carré d'une multiplication en exponentiation modulaire).
- **DPA (Differential Power Analysis)** : analyse statistique de nombreuses traces de consommation pour un grand nombre d'entrées, permettant de retrouver des bits de clé même en présence de bruit.

Ces attaques concernent principalement des dispositifs physiques (cartes à puce, modules matériels de sécurité, objets connectés) où l'attaquant a un accès physique ou de proximité.

5. Attaques par cache (cache-timing attacks)

Exploitent les différences de temps d'accès entre données présentes ou absentes du cache processeur. Des implémentations logicielles d'AES utilisant des tables précalculées (T-tables) indexées par des bits de la clé ont ainsi été attaquées avec succès, motivant l'adoption d'instructions matérielles dédiées (AES-NI) exécutant le chiffrement en temps constant indépendamment de la clé.

6. Attaques sur les générateurs de nombres aléatoires

Un générateur de nombres pseudo-aléatoires (PRNG) de mauvaise qualité ou mal initialisé (faible entropie au démarrage, graine prévisible) compromet directement toute construction cryptographique qui en dépend (génération de clés, de nonces, de vecteurs d'initialisation). Cas réel documenté : une faille dans le générateur aléatoire d'une distribution Linux (Debian, 2006-2008) a réduit l'espace des clés SSH générées à quelques dizaines de milliers de valeurs possibles, les rendant énumérables.

7. Démarche défensive face aux canaux auxiliaires

- Utiliser des implémentations cryptographiques éprouvées et auditées (bibliothèques reconnues) plutôt que réimplémenter soi-même des primitives sensibles.
- Privilégier les implémentations en temps constant pour toute opération manipulant un secret.
- S'assurer d'une source d'entropie fiable pour toute génération aléatoire cryptographique (`/dev/urandom`, CSPRNG dédié).
- Pour les dispositifs physiques sensibles, envisager des contre-mesures matérielles (masquage, bruit ajouté).

À retenir

- Les attaques par canaux auxiliaires exploitent l'implémentation, pas l'algorithme : elles rappellent qu'une preuve mathématique de sécurité ne suffit pas.
- Le padding oracle est une attaque pratique majeure contre les usages naïfs de CBC, à l'origine de plusieurs failles réelles dans TLS/SSL.
- Un générateur aléatoire de mauvaise qualité peut annuler la sécurité de tout système cryptographique qui en dépend, quelle que soit la robustesse théorique de l'algorithme utilisé.